

Infografía

Módulo 7 — Godot 4 • Sesión 3

Física y colisiones

UPB • Gestión I/2026

Plan de la clase

1. **Pachinko** — opener: los 3 tipos de cuerpo trabajando juntos.
2. **Los 4 tipos de cuerpo** — Static, Character, Rigid, Area.
3. **Gravedad y salto** — `CharacterBody2D` en side-view (completamos en vivo).
4. `Area2D` — pickups y hazards con `body_entered`.
5. **Capas y máscaras** — el "aha" del día.
6. **Ejercicios** — doble salto + enemigo patrullando.

En la sesión 2 movimos un vector (top-down, sin gravedad). Hoy el mundo TIENE física: las cosas caen, rebotan, detectan. Cambiamos el paradigma.

1. Pachinko

los 3 tipos de cuerpo, en acción

¿Qué estás viendo?

Demo: `01_pachinko.tscn` (F5 — es la escena principal).

- **Clavos** que no se mueven (las bolas chocan contra ellos).
- **Bolas** que caen con gravedad y rebotan.
- **Cajas** abajo que **detectan** sin chocar (cuentan cada bola que entra).

Tres tipos de cuerpo distintos. Cada uno hace su trabajo.

No es magia: cada clavo, cada bola y cada caja es un nodo con un tipo de body distinto. Vamos a ver cuáles son.

2. Los 4 tipos de cuerpo

la pieza más importante de hoy

La pregunta clave

“ ¿Quién decide cómo se mueve este objeto? ”

cuatro tipos — la diferencia es QUIEN decide su movimiento

StaticBody2D

no se mueve.
los demas chocan contra el.

*piso, paredes, plataformas,
clavos del pachinko.*

CharacterBody2D

vos definis velocity.
move_and_slide() lo desliza.

*jugador, enemigo,
cualquier personaje "controlado".*

RigidBody2D

el motor lo mueve.
gravedad, masa, rebote, impulsos.

*bolas del pachinko, cajas que caen,
escombros, proyectiles "fisicos".*

Area2D

NO choca. solo detecta.
emite body_entered / area_entered.

*monedas, puas, zonas de peligro,
triggers, hurtbox de un enemigo.*

La elección de body type es la primera decisión que tomas al modelar un objeto físico.

En el pachinko, ¿quién es quién?

Pieza	Body type	Quién la mueve
Clavos	StaticBody2D	nadie (es estática)
Bolas	RigidBody2D	el motor (gravedad, rebote)
Cajas	Area2D	nadie (no se mueve, solo detecta)

Y el jugador del módulo (que viene en la próxima parte) será un **CharacterBody2D**: tú decides su velocidad, el motor resuelve los choques.

Mismo motor, cuatro contratos distintos. El cuarto tipo es el que ya usamos en sesión 2.

3. Gravedad y salto

CharacterBody2D, ahora con peso

La diferencia con la sesión 2

Sesión 2 (top-down): si soltabas todas las teclas, el jugador se quedaba quieto.

Sesión 3 (side-view): si no aplicas gravedad, el jugador queda **flotando en el aire**.

La gravedad no aparece sola — la pones tú en el código.

```
const GRAVEDAD: float = 1200.0    # px/s2

func _physics_process(delta: float) -> void:
    velocity.y += GRAVEDAD * delta
    # ... mover horizontal, etc ...
    move_and_slide()
```

En Godot la Y crece hacia abajo: sumar a velocity.y = caer. Restar = subir.

Saltar = un impulso vertical

Saltar es **una sola línea**: poner `velocity.y` a un valor negativo grande.

```
if Input.is_action_just_pressed("saltar"):
    velocity.y = VELOCIDAD_SALTO # ej: -520
```

Después la gravedad va sumando a `velocity.y` cada frame, así que la `y` sube (porque arranca muy negativa), llega a 0 en el punto más alto, y empieza a bajar.

Pero hay un problema: ¿qué pasa si seguimos apretando "saltar" en el aire?

`is_on_floor()` — el chequeo clave

`CharacterBody2D` recuerda si en el último `move_and_slide()` tocó el piso.

```
if is_on_floor() and Input.is_action_just_pressed("saltar"):
    velocity.y = VELOCIDAD_SALTO
```

Solo saltamos si **ya** estamos en el piso. En el aire, la tecla no hace nada.

Combinas dos cosas: el estado físico (`is_on_floor`) + el input (`just_pressed`).

Si quieres permitir doble salto, cambias esta condición. Lo ves en ex1.

Placeholder: 02_gravedad_y_salto.tscn

Lo completamos en vivo (3 TODOs).

```
func _physics_process(delta: float) -> void:
    # TODO 1: velocity.y += GRAVEDAD * delta
    # TODO 2: leer input -> velocity.x
    # TODO 3: saltar si is_on_floor() y just_pressed("saltar")
    move_and_slide()
```

F6 sobre 02_gravedad_y_salto.tscn. La escena ya tiene piso, paredes y plataformas — el código es lo único que falta.

4. Area2D

detectar sin chocar

Area2D: el "trigger" de Godot

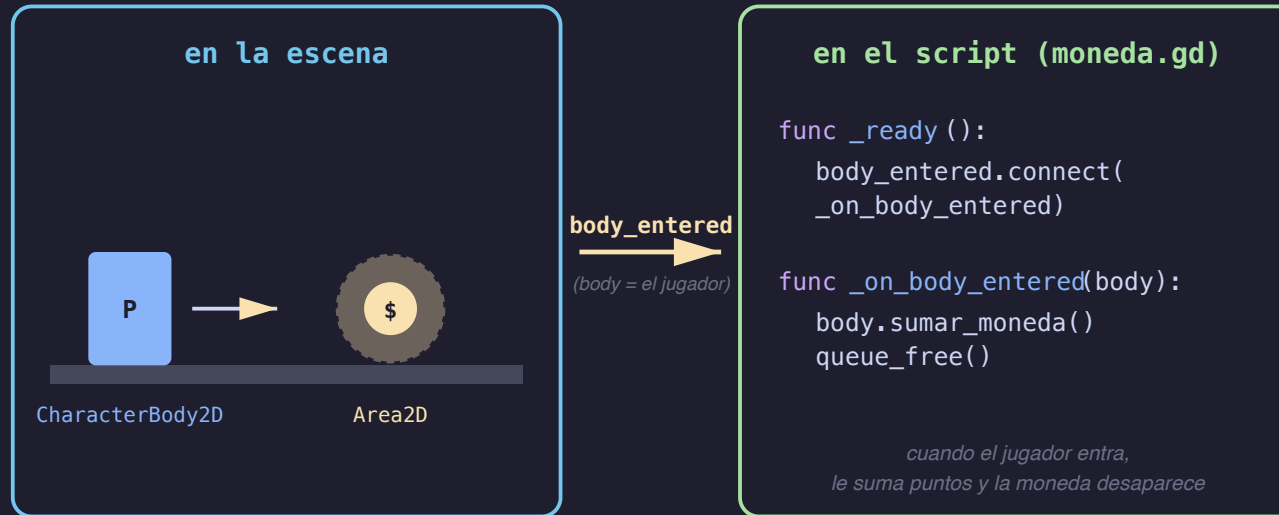
Hasta ahora todos los cuerpos **bloqueaban** al jugador. Un `Area2D` es distinto: **no choca, detecta**.

- El jugador atraviesa el área sin frenar.
- Cuando entra, el área dispara `body_entered(body)`.
- Cuando sale, dispara `body_exited(body)`.

Bloquear = StaticBody/CharacterBody/RigidBody. Detectar = Area2D.

El patrón

Area2D no choca — emite una señal cuando un cuerpo entra. El script reacciona.



El Area2D no hace nada por sí solo. Tu script conecta la señal y decide qué pasa.

Moneda recolectable

```
extends Area2D

func _ready() -> void:
    body_entered.connect(_on_body_entered)

func _on_body_entered(body: Node) -> void:
    body.sumar_moneda() # método del jugador
    queue_free()       # la moneda desaparece
```

Si el jugador "entra" a la moneda (sus collision boxes se tocan), la señal dispara. La moneda no detiene al jugador — solo escucha.

Púa / zona de daño

Mismo patrón que la moneda, distinto efecto:

```
extends Area2D

func _ready() -> void:
    body_entered.connect(_on_body_entered)

func _on_body_entered(body: Node) -> void:
    body.recibir_dano()
    # ojo: NO queue_free – la púa queda como peligro permanente
```

Mismo nodo (Area2D), misma señal (body_entered), distinto handler. La señal es el contrato; el handler decide la lógica.

Demo: 03_pickups_y_hazards.tscn

Completamos los dos handlers en vivo:

Archivo	TODO
scripts/moneda.gd	body.sumar_moneda() + queue_free()
scripts/pua.gd	body.recibir_dano()

El jugador ya tiene los métodos `sumar_moneda()` y `recibir_dano()` (no los completamos hoy; tienen lógica de respawn y HUD que ya funciona).

F6 sobre la escena. Antes de completar los TODOs: el jugador pasa por encima de monedas y púas sin que pase nada. Después: monedas suman al HUD; púa respawnea.

5. Capas y máscaras

el "aha" del día

El problema

Imagina que tienes en tu juego:

- Jugador, enemigos, balas del jugador, balas de enemigos.
- Paredes que bloquean a todos.
- Power-ups que solo el jugador puede recoger.

¿Cómo le dices al motor **qué choca con qué?**

No quieres que las balas del jugador se choquen entre sí. No quieres que los enemigos recojan power-ups. La respuesta de Godot: capas y máscaras.

Dos conceptos, dos checkboxes

Cada cuerpo (CharacterBody2D, RigidBody2D, Area2D, etc.) tiene **dos bitsets** en el Inspector:

- `collision_layer` — **lo que SOY** (en qué capas vivo).
- `collision_mask` — **lo que DETECTO / ESCUCHO** (en qué capas escaneo).

Cuando dos cuerpos chequean si chocan, Godot pregunta:

“ ¿La `mask` del cuerpo A toca alguna capa que está en la `layer` del cuerpo B? ”

Si sí: hay colisión. Si no: se ignoran.

Las capas de nuestro proyecto

1 – mundo	(piso, paredes, plataformas)
2 – jugador	
3 – muro	(pilares interiores que el fantasma atraviesa)
4 – recolectable	(monedas)
5 – hazard	(púas)
6 – enemigo	
7 – pelota	(bolas del pachinko)

*Cada capa es **UN BIT**. Una colisión es un AND entre dos bitsets. Eso es todo.*

El demo del fantasma

`04_layers_fantasma.tscn`: dos personajes en una escena.

- **Player** — controlado por WASD/Espacio.
- **Fantasma** — deambula horizontalmente con una IA mínima.

Ambos tienen:

```
collision_layer = jugador (2)  
collision_mask  = mundo (1) + muro (4) = 5
```

Chocan contra el piso (`mundo`) y contra los pilares interiores (`muro`).

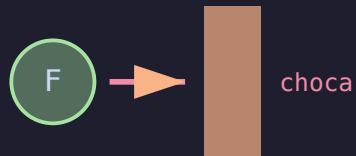
El "aha" — el toggle de un bit

collision_mask = "que cosas VEO" — destildar un bit hace al fantasma ignorar esa capa

mask por defecto

- ☒ bit 1 — mundo
- ☐ bit 2 — jugador
- ☒ bit 3 — muro

fantasma frente a un pilar:

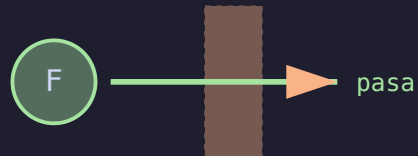


*mask incluye muro
→ el motor calcula el choque*

mask con muro destildado

- ☒ bit 1 — mundo
- ☐ bit 2 — jugador
- ☐ bit 3 — muro ← *destildado*

fantasma frente a un pilar:



*mask ignora muro
→ el motor no chequea esa capa*

Inspector → Fantasma → CollisionObject2D → Mask → destildar el bit "muro". F6.

¿Qué pasa?

- **Antes** del toggle: fantasma camina hasta un pilar, se choca, gira.
- **Después** del toggle: fantasma atraviesa los pilares como si no existieran. **Pero sigue acotado** por el piso y las paredes externas (capa `mundo`, sigue en la mask).

No cambió nada del código. Cambió un bit en el Inspector.

Esto es lo que hace al sistema de capas tan poderoso: la lógica de "qué interactúa con qué" vive en datos, no en código.

El mismo principio en todo el proyecto

Caso	layer (soy)	mask (escaneo)
Pared	mundo	— (estática, no escanea)
Player	jugador	mundo + muro
Moneda (Area2D)	recolectable	jugador
Púa (Area2D)	hazard	jugador
Bola del pachinko	pelota	mundo + pelota

*Lee cualquier fila como "soy X, escaneo Y". La moneda **es** recolectable, **escanea** al jugador. Cuando el jugador la toca, la moneda recibe `body_entered`.*

6. Ejercicios

para practicar después de clase

ex1 – doble salto

Objetivo: permitir saltar HASTA 2 veces antes de tocar piso.

```
const MAX_SALTOS: int = 2
var saltos_usados: int = 0

# en _physics_process:
if is_on_floor():
    saltos_usados = 0

# TODO: cambiar la condición del salto
if is_on_floor() and Input.is_action_just_pressed("saltar"):
    velocity.y = VELOCIDAD_SALTO
```

Concepto: estado en una variable. Misma forma que el dash de sesión 2.

ex2 – enemigo patrullando

Objetivo: enemigo que camina ida y vuelta en un rango; daña al jugador al tocarlo.

3 TODOs:

1. Moverse horizontalmente.
2. Dar vuelta al llegar a los límites (`x_min` / `x_max`).
3. Conectar `body_entered` del **Hurtbox** (`Area2D` hijo) y llamar `body.recibir_dano()`.

Concepto clave: separar movimiento (`CharacterBody2D`) de detección (`Area2D` hijo). Es el patrón Hurtbox de cualquier juego 2D.

Resumen

lo que tienes que llevarte hoy

En una página

1. **4 tipos de cuerpo:** Static (no se mueve), Character (tú lo mueves), Rigid (el motor lo mueve), Area (no choca, detecta).
2. **Gravedad** = sumar a `velocity.y` cada frame; **salto** = `velocity.y = -X` si `is_on_floor()`.
3. `Area2D` + `body_entered` = pickups, hazards, triggers, hurtboxes.
4. **Capas y máscaras:** `layer` = lo que SOY, `mask` = lo que DETECTO. Configuradas en el Inspector, no en código.
5. **Patrón Hurtbox:** un `CharacterBody2D` para mover + un `Area2D` hijo para el "daño" — separar movimiento de detección.

Sesión 4: animaciones y máquinas de estado. La idea de "estado en una variable" (saltos_usados, dash, ataque) reaparece como `AnimationTree`.